

Technical Specification: Project AEGIS-AML

An Agentic Human-in-the-Loop Governance Engine for Financial Integrity

1. Executive Summary

AEGIS-AML is a next-generation Anti-Money Laundering (AML) framework that moves beyond static threshold-based alerts. It integrates **Deterministic Guardrails**, **Gradient Boosted ML**, and **Graph Intelligence** into a unified agentic workflow. The system features a unique **Human-in-the-Loop (HITL)** reinforcement layer, allowing compliance officers to "teach" the model and simulate regulatory changes using natural language.

2. The Problem: The Compliance Trilemma

Modern AML systems struggle to balance three conflicting goals:

1. **High Recall:** Catching sophisticated money laundering (mules/layering).
 2. **Low False Positives:** Reducing the 95% noise rate that drains human resources.
 3. **Explainability:** Meeting "Right to Explanation" regulatory requirements.
-

3. Layered Intelligence Architecture

Layer 1: RAW Agent (Deterministic Guardrails)

- **Engine:** Hardcoded logic & Regex.
- **Logic:** Sanctioned country lists, velocity spikes (e.g., \$ > 5\$ transactions in 1 hour), and fixed thresholds (\$ > ₹10L\$).
- **Role:** Acts as the "Safety Valve" to ensure immediate regulatory compliance.

Layer 2: SAR Agent (Probabilistic ML)

- **Model:** XGBoost / LightGBM.
- **Feature Engineering:** * Temporal: Time-of-day entropy.
 - Behavioral: Deviation from mean transaction volume (\$Z-score\$).
 - Demographic: Account age vs. activity ratio.
- **Output:** A probability score (\$P_{\text{risk}}\$) and SHAP values for local explainability.

Layer 3: Graph Intelligence (Relational Analysis)

- **Engine:** NetworkX / Neo4j.
 - **Metrics:** * **Community Detection:** Finding "Mule Clusters."
 - **Eigenvector Centrality:** Identifying high-influence nodes in a laundering ring.
 - **Shortest Path:** Calculating distance to confirmed "Bad Actors."
-

4. The Agentic Orchestrator

We utilize an **Agentic Framework** (e.g., LangGraph) to act as a "Chief of Staff."

- **Dynamic Weighting:** The orchestrator calculates the final risk score:
$$Risk_{final} = (w_1 \cdot Layer_1) + (w_2 \cdot Layer_2) + (w_3 \cdot Layer_3)$$
 - **The "Analyst" Agent:** If a high-risk score is generated, this agent queries the Graph layer and ML layer to write a **Narrative Brief** for the human officer.
-

5. Innovation Pillars

A. Natural Language Policy Engine (NLPE)

Allows non-technical officers to input rules: *"Flag any account younger than 30 days transferring more than ₹50k to a high-risk region."*

- **Process:** The workflow involves transforming the LLM output into Structured JSON, which then feeds into SQL/Python Logic for execution by the Backtesting Engine.
- **Value:** Instant deployment of new policies without code changes.

B. Human-in-the-Loop (HITL) Reinforcement

- **The Reinforcement Cache:** Officer decisions (True/False Positive) are stored as "Feedback Vectors."
- **Dynamic Calibration:** Instead of full retraining (which is slow), the system adjusts the **Orchestrator Weights** (w_n) based on recent officer feedback to suppress future false positives.

C. Explainability (XAI)

Every alert includes an **Evidence Bundle**:

- **Visual:** Sub-graph showing the mule network.
 - **Textual:** "Model flagged this due to 89% similarity to 'Layering' behavior patterns."
-

6. Consumer Feature: Scam Exposure Shield

A B2C API that democratizes the Graph Intelligence layer.

- **Input:** UPI ID / Account Number.
- **Logic:** Calculates the **Network Proximity** to suspicious clusters.
- **UX:** Returns a simple "Trust Score" (1-10) to prevent citizens from falling for scams before they transfer money.

7. Technical Stack

Component	Technology
Backend	Python (FastAPI)
AI Agents	LangGraph / CrewAI
ML Models	XGBoost / Scikit-Learn
Graph Database	NetworkX / Neo4j
Storage	PostgreSQL (Relational) + Redis (Cache)
Frontend	Streamlit (Admin) + React (Consumer)